

Lecture 3: Adapter Design Pattern

Group Activities for Students

Software Design Patterns - complete one activity after each video section

Instructions: Work in a group of five. Submit one shared answer per group. Write the names of all group members. You may answer on this sheet or on a laptop, but your group must be ready to show the completed work.

Group number:	5	SE group(s):	2536
Student 1:	Nodir Mukhammedov	Student 2:	Nurken Mamay
Student 3:	Nauryzbay Kinayatov	Student 4:	
Student 5:			

Activity 1 - Find the Incompatibility

Watch first: Slides 1-6.

Scenario

A university has an old SMS notification system:

```
LegacySMS  
sendSMS(phoneNumber, message)
```

A new application expects every notification service to provide:

```
Notification send(recipient,  
message)
```

Your team wants to use LegacySMS without changing the old class.

1. What exactly is incompatible?

The method names (send vs. sendSMS) and the first parameter names (recipient vs. phoneNumber) differ, preventing direct calls.

2. What should the Target Interface be?

The Notification interface with the send(recipient, message) method.

3. What is the Adaptee?

The LegacySMS class.

4. What should the Adapter do?

Implement Notification, receive the send(recipient, message) call, and delegate it to legacySMS.sendSMS(phoneNumber, message) internally.

Complete the translation:

```
Client calls: notification.send(recipient, message)
Adapter translates it into: legacySMS.sendSMS(recipient, message)
```

Activity 2 - Identify the Four Components

Watch first: Slides 7-14.

Scenario

Your application expects:

```
interface PaymentProcessor { processPayment(amount)
}
```

But an external payment library provides:

```
class FastPay {
  makeTransaction(amount)
}
```

Use these names: ShoppingApp, PaymentProcessor, FastPayAdapter, FastPay.

Role	Your answer
Client	ShoppingApp
Target	PaymentProcessor
Adapter	FastPayAdapter
Adaptee	FastPay

Draw the request flow from the Client to the real service:

```
ShoppingApp → processPayment(amount) FastPayAdapter → makeTransaction(amount) FastPay
```

Would you use an Object Adapter or a Class Adapter here? Explain why.

Object Adapter. Languages like Java do not support multiple class inheritance. Using composition (holding an instance of FastPay inside FastPayAdapter while implementing PaymentProcessor) provides loose coupling and higher flexibility.

Activity 3 - Complete the TurkeyAdapter

Watch first: Slides 15-17.

The client expects a Duck. The real object is a Turkey. Complete the Adapter.

```

class TurkeyAdapter implements Duck { Turkey turkey;

TurkeyAdapter(Turkey turkey) {
this.turkey = turkey;

} public void quack()

{    turkey.gobble();

} public void fly()

{

    for (int i = 0; i < 5; i++) {
turkey.fly();
    }
}
}

```

Why does the client accept TurkeyAdapter where it expects a Duck?

Because TurkeyAdapter implements the Duck interface, fulfilling the polymorphic type contract expected by the client.

In one sentence, explain what the Adapter is translating in this example.

The adapter translates duck quacks into turkey gobbles and compensates for a duck's longer flight by calling the turkey's short flight multiple times.

Activity 4 - Square Peg into a Round Hole

Watch first: Slides 18-21.

The round hole has radius 5. The Adapter converts a square peg width into an equivalent radius using:

```
equivalent radius = width x sqrt(2) / 2
```

Calculate the adapted radius and decide whether each square peg fits.

Square peg width	Adapted radius	Fits radius-5 hole?
4	$\approx 2.83 (2\sqrt{2})$	Yes
6	$\approx 4.24 (3\sqrt{2})$	Yes
8	$\approx 5.66 (4\sqrt{2})$	No
10	$\approx 7.07 (5\sqrt{2})$	No

Which object is being adapted?

SquarePeg.

What interface does the Adapter make it look like?

RoundPeg (providing getRadius())

Does the RoundHole class need to be changed? Explain.

No, RoundHole continues to interact with RoundPeg instances as designed, adhering to the Open/Closed Principle.

Activity 5 - Build Your Own Adapter

Watch first: Slides 22-25.

Problem

Your application expects:

```
interface MediaPlayer {  
    void play(String file);  
}
```

But you need to use an old library:

```
class LegacyAudioPlayer { void startAudio(String  
    filename) { ... } }
```

You are not allowed to modify either existing class.

1. Identify the four Adapter-pattern roles.

Client	The application calling audio playback.
Target	MediaPlayer
Adapter	AudioPlayerAdapter (or LegacyAudioPlayerAdapter)
Adaptee	LegacyAudioPlayer

2. Write the Adapter in Java or clear pseudocode.

```

public class AudioPlayerAdapter implements MediaPlayer {
    private LegacyAudioPlayer legacyPlayer;

    public AudioPlayerAdapter(LegacyAudioPlayer legacyPlayer) {
        this.legacyPlayer = legacyPlayer;
    }

    @Override
    @Override
    public void play(String file) {
        legacyPlayer.startAudio(file);
    }
}

```

3 Draw the request flow from the client to the legacy player.

```

      play(file)           startAudio(file)
Client → AudioPlayerAdapter → LegacyAudioPlayer

```

4. Why is Adapter appropriate here instead of changing the existing classes?

Modifying LegacyAudioPlayer could break existing legacy systems or may be impossible if it comes from an external precompiled library. The Adapter adheres to the Open/Closed Principle by allowing integration without changing existing source code.

Before you finish: Make sure every group member can explain the Target, Adaptee, Adapter, and the translation being performed.